

ML Engineer — Detailed Guide

Retail Demand Forecasting (Walmart M5, Hierarchical) · Team Study Material

1. What machine learning means here

ML — Machine Learning. *Instead of a programmer writing the rules, the computer finds the rules from examples. You show it thousands of situations where you already know the answer, and it learns a function that maps situation → answer.*

In our project:

- **the situation** = 38 numbers describing a product on a day (the features)
- **the answer** = how many units it actually sold
- **learning** = adjusting an internal model until its guesses are close
- **using it** = feeding in a *future* situation and reading off the guess

AI vs ML. *AI (Artificial Intelligence) is the broad field of machines doing things that seem intelligent. ML is the part of AI that learns from data. Our forecasting model is ML. The Gemini assistant is a different kind of AI — it explains our numbers, it does not produce them.*

Why forecasting is a machine-learning problem

You could try writing rules by hand: "if it sold 3 last week, predict 3."

That fails immediately, because demand depends on weekday, season, price, promotions, SNAP days, how long since it last sold, which store, which category — and those interact. With 30,490 products you cannot hand-write rules for each.

So instead we hand the machine 12.8 million examples of "here is the situation, here is what actually happened" and let it find the pattern.

Supervised learning. *Learning from examples where the correct answer is known. Ours is supervised. The correct answers come from history.*

2. What LightGBM is, from scratch

LightGBM = Light Gradient Boosting Machine, an open-source library from Microsoft. To understand it, build it up in three steps.

Step 1 — a decision tree. A flowchart of yes/no questions:

```
is rolling_mean_28 < 0.2 ?
  yes -> is days_since_last_sale > 30 ?
    yes -> predict 0.02
    no -> predict 0.15
  no -> is wday == Saturday ?
    yes -> predict 3.8
    no -> predict 2.1
```

Each question splits the data; each leaf holds a prediction.

Step 2 — boosting. One tree is weak. So build a tree, see where it is wrong, then build a **second** tree whose job is to predict the **remaining error**. Then a third for what is still wrong. Add them all up.

Boosting. Building models in sequence, each one correcting the mistakes of the ones before. "Gradient" boosting means each new tree is fitted to the direction that most reduces the error.

Step 3 — "Light". LightGBM grows trees leaf-wise (always splitting the leaf that reduces error most) and bins continuous features into buckets. That makes it fast enough to train on tens of millions of rows on a normal machine — which is exactly our situation.

Why we chose it: it handles categorical features natively (`store_id`, `dept_id`), it is fast on our data size, it copes with the extreme number of zeros, and it is deterministic when seeded — so our results reproduce exactly.

3. What the Tweedie objective is, and why it matters

Objective function (loss function). *The definition of "wrong" that the model is trying to minimise. Change the objective and you change what the model considers a good prediction.*

The default objective for numeric prediction is squared error, which quietly assumes the target is roughly bell-shaped around its average. **Our target is not remotely bell-shaped.** 68% of values are exactly zero, and the rest are small positive integers with occasional spikes.

Tweedie is a family of distributions designed for exactly this shape: a solid lump of probability at zero, plus a positive skewed tail. It is the standard choice for insurance claims — most policies claim nothing, a few claim a lot — and retail demand behaves the same way.

The family has a parameter, `tweedie_variance_power`, usually between 1 and 2:

- **at 1** it behaves like a Poisson distribution (counts, no zero-inflation)
- **at 2** it behaves like a Gamma distribution (positive, no exact zeros)
- **between** you get the mixture: exact zeros *and* a positive tail

VERIFIED: we use `tweedie_variance_power = 1.1` — *close to the Poisson end, which suits mostly-small counts with a very large mass at zero.*

One consequence to remember: a Tweedie model predicts an **expected value**, so it happily outputs 0.6 units. That is not a bug. It means "about 0.6 units a day on average" — over 28 days that is roughly 17 units, which is a real, actionable number.

4. Our two models

We do not train one model. We train **two architecturally different** ones and combine them. Both are **global** models — a single model across all 30,490 series, not 30,490 separate models.

Global model. One model for every series. It sees `store_id` and `item_id` as features, so it can learn shared patterns (weekends are busy) while still distinguishing individual products. With 30,490 series, most of which sell rarely, a per-series model would have far too little data each.

Member A — DIRECT

Property	Verified value
Algorithm	LightGBM
Objective	tweedie, tweedie_variance_power = 1.1
Features	38 (CHAMPION_FEATURES)
Boosting rounds	400
Learning rate	0.05
num_leaves	128
min_data_in_leaf	100
feature_fraction / bagging_fraction	0.8 / 0.8, bagging_freq 1
lambda_l2	1.0
max_cat_threshold	32
Seed	42, deterministic = True, force_row_wise = True
Training design	15 origins x 30,490 series x 28 days = 12,805,800 rows

It predicts all 28 days **in one shot**, using horizon as a feature.

Member B — RECURSIVE

Property	Verified value
Algorithm	LightGBM, one-step-ahead, rolled forward 28 times
Objective	tweedie, tweedie_variance_power = 1.1
Features	32 (REC_COLS_V5)
Boosting rounds	400
Training design	420 daily origins, d_1521 ... d_1940, horizon = 1
Seed	42

It predicts tomorrow, feeds that prediction back in as history, predicts the day after, and repeats 28 times.

What the hyperparameters mean

Hyperparameter. A setting **you** choose before training, as opposed to something the model learns.

Setting	Plain English
num_leaves = 128	how complex each tree may be. Bigger = can capture more, risks memorising

Setting	Plain English
learning_rate = 0.05	how big a step each new tree takes. Small = slower but steadier
400 rounds	how many trees. With lr 0.05, 400 trees is a deliberate pairing
min_data_in_leaf = 100	a leaf must be backed by ≥ 100 examples — stops it inventing rules from 3 rows
feature_fraction = 0.8	each tree sees a random 80% of features — adds diversity
bagging_fraction = 0.8	each tree sees a random 80% of rows — same idea
lambda_l2 = 1.0	a penalty on extreme leaf values; keeps predictions sane
seed = 42, deterministic	the same run gives the same model, every time

Overfitting. When a model memorises the training data instead of learning the pattern — brilliant on data it has seen, poor on anything new. min_data_in_leaf, feature_fraction, bagging_fraction and lambda_l2 all exist to prevent it.

5. Training and validation split

We do **not** split randomly. Time-series data must be split by **time**, or the model would train on the future and be tested on the past.

Block	Days	Dates
Training	d_1 ... d_1913	2011-01-29 ... 2016-04-24
Validation	d_1914 ... d_1941	2016-04-25 ... 2016-05-22
Final forecast	d_1942 ... d_1969	2016-05-23 ... 2016-06-19

Forecast origin. *The last day whose sales the model may see. Everything after it must be predicted, never observed.*

Every training origin obeys the rule `origin + horizon <= validation_origin`, so no training example ever spans into the evaluation window.

6. What training produces — the artefacts

Model artefact. *The saved file containing everything learned. It is what you ship; you do not retrain in production.*

File	Which member
model_11_blend_direct_final_forecast.txt	direct (38 features)
model_12_blend_recursive_shape_final.txt	recursive (32 features)

Canonical location: 03_RESEARCH/models/champion/. A certified copy plus its manifest:
02_DOCUMENTATION/02_MODEL/FROZEN_CHAMPION/.

They are LightGBM text files — a readable description of every tree.

7. Why this model was selected

86 experiments are recorded in 03_RESEARCH/experiments/registry/, one JSON file each. The champion is exp_78_blend_final_forecast.

Verified results on the primary validation window (853,720 predictions):

Model	RMSE	MAE
Direct member alone	2.1211	1.0293
Recursive member alone	2.1185	1.0689
Blend 0.60 / 0.40	2.0929	1.0395
Naive "repeat the 28-day average"	2.2430	—
Predict zero everywhere	3.9161	—

The blend beats both members. It is not a tuning trick — the two members are architecturally different, and their residual correlation is **0.9488**, low enough that combining them cancels some error.

Why we stopped. The research concluded RMSE below 2.0 is not reachable with the information in this dataset, on three independent lines of evidence: six architecturally different models produce residuals correlated at **0.9897**; every feature family was tested and none helped further; and an oracle that already knows each series' true 28-day mean still only reaches **1.9818**. What would improve this is **more information** — a promotions calendar, inventory records, footfall — none of which exists in this data.

8. The frozen model

Frozen. The model is final. It is not retrained, re-tuned, re-seeded or replaced. The application consumes its output; it never modifies it.

Why freeze? Because every accuracy figure the product displays was measured against *these exact weights*. Retrain the model and every number on every page becomes a claim about a model that no longer exists.

How the freeze is enforced, in code:

- 02_DOCUMENTATION/02_MODEL/MODEL_FREEZE.md states the specification.
- CHAMPION_MANIFEST.json records the SHA-256 hash of both model files.
- 01_PROJECT/backend/tests/test_integrity.py re-hashes them and fails if they

differ; it also checks the served forecast matches the frozen file row for row, and that the backtest still reproduces RMSE 2.0929 / MAE 1.0395.

- The API mounts model files **read-only** and runs as a non-root user.

VERIFIED: a protected-artefact manifest covers **522 files**; the last comparison reported **0 deleted, 0 modified**.

Training vs inference

	Training	Inference
What happens	The model learns from examples with known answers	The saved model is applied to new situations
When	Once, in research	Every time a forecast is needed
Output	A model file	Predictions
In our project	Done, frozen, not repeated	Reproducible on demand

Inference. Using a trained model to make predictions. No learning happens.

Verified timings (from 01_PROJECT/backend/README.md): applying the direct member to all 853,720 rows takes **4.1 s**; the recursive rollout takes about **29 s**; a complete blended re-forecast is about **33 s**.

10. Verification, and how the model is used

POST /api/v1/inference/verify reloads both frozen models, rebuilds the features, re-runs the blend and compares against the shipped forecast.

VERIFIED RESULT: `max_abs_diff = 0.000e+00` **across all 853,720 predictions.** *The live model reproduces the shipped forecast exactly.*

Also verified: the leakage suite passes 46/46; the corruption test (all post-origin sales overwritten with 9999) leaves every feature bit-identical; and during the Use Case 11 audit all four validation windows were **retrained from scratch** and reproduced the same figures to four decimals.

11. Common misunderstandings

"The model is trained when the app runs." No. Training happened once. The app loads a saved file.

"Predicting 0.6 units is a bug." No. It is an expected value — about 17 units over 28 days.

"You should have used a neural network / ARIMA." We tested six architecturally different models; their residuals correlate at 0.9897, meaning they make the *same* mistakes. Model choice is not the bottleneck — information is.

"Higher accuracy is always better." Chasing a lower RMSE past the oracle bound (1.9818) would mean fitting noise.

"The AI assistant makes the forecast." No. The forecast comes from LightGBM. Gemini only explains it.

12. How to present this

"My part is the model. We use LightGBM — a gradient-boosting library that builds lots of small decision trees, where each tree corrects the errors of the ones before it.

The important choice is the objective — the definition of 'wrong' the model minimises. We use Tweedie with variance power 1.1, because 68% of our data is exactly zero. Normal squared error assumes a bell curve, which our data is nothing like. Tweedie is built for data with a big spike at zero plus a positive tail — it's what insurance uses for claims.

We actually train two models. A direct one that predicts all 28 days at once using 38 features, and a recursive one that predicts one day and feeds its own prediction back in, using 32 features. On its own the direct model gets RMSE 2.1211 and the recursive 2.1185, but blended at 60/40 we get 2.0929 — better than either, because they're architecturally different and make somewhat different mistakes.

Everything is seeded and deterministic, and the model is frozen: the hashes of both model files are recorded and a test fails if they ever change. And we can prove it still works — there's an endpoint that reloads the models and re-runs the whole forecast, and it matches the shipped numbers with a maximum absolute difference of exactly zero."

13. Likely judge questions

Beginner

Q: What model did you use? A: LightGBM — gradient-boosted decision trees — with a Tweedie objective. Two of them, blended 60/40.

Q: What does the model actually predict? A: Units sold, per product, per store, per day, for the next 28 days.

Q: How long does training take? A: Training is done and frozen. Re-running the finished model over all 853,720 predictions takes about 33 seconds for the full blend.

Technical

Q: Why Tweedie instead of squared error? A: 68% of our target values are exactly zero. Squared error assumes a roughly bell-shaped target. Tweedie models a point mass at zero plus a positive skewed tail — the actual shape of retail demand. We use variance power 1.1, near the Poisson end.

Q: Why a global model rather than one per series? A: 30,490 series, most selling rarely — per-series models would starve for data. A global model shares patterns across products while still seeing `store_id` and `item_id` as features.

Q: How do you prevent overfitting? A: `min_data_in_leaf = 100`, `feature_fraction` and `bagging_fraction` at 0.8, `lambda_l2 = 1.0`, and a modest learning rate of 0.05 over 400 rounds. And the split is by time, never random.

Q: Why should I believe your numbers? A: Fixed seed and `deterministic = True`, so runs reproduce. The blend was weighted on an inner window that ends before the evaluation window. All four validation windows were retrained from scratch during an audit and reproduced to four decimals. And live re-inference matches the shipped forecast with `max_abs_diff = 0.000e+00`.

Q: Could you get RMSE below 2.0? A: Our evidence says not with this data. Six different architectures produce residuals correlated at 0.9897, and an oracle that already knows each series' true 28-day mean only reaches 1.9818. The limit is information, not modelling.

14. Glossary

Term	Meaning
ML	Machine Learning — learning rules from examples
Supervised learning	Learning from examples with known answers
LightGBM	Light Gradient Boosting Machine — the library we use
Decision tree	A flowchart of yes/no splits ending in a prediction
Boosting	Building models in sequence, each fixing the previous errors
Objective / loss	The definition of "wrong" being minimised
Tweedie	A distribution family for data with many zeros plus a positive tail
Hyperparameter	A setting chosen before training
Overfitting	Memorising training data instead of learning the pattern
Global model	One model across all series
Model artefact	The saved trained model file
Inference	Using a trained model to predict; no learning happens
Forecast origin	Last day whose sales the model may see
Deterministic	Same inputs and seed produce the same model every time
Frozen	Final; never retrained or replaced